

1. (a) For the following code given for events, discuss about the displayed output. Briefly explain your result (point wise) by mentioning proper reason.

```

1 module events_example;
2   event ev_A, ev_B, ev_C;
3   initial begin
4     fork
5       begin
6         #12
7         $display($time, "\t triggering The Event ev_A");
8         ->ev_A;
9       end
10      begin
11        #5
12        $display($time, "\t triggering The Event ev_C");
13        ->ev_C;
14      end
15      begin
16        #8
17        $display($time, "\t triggering The Event ev_B");
18        ->ev_B;
19      end
20      begin
21        $display($time, "\t waiting for the Event to trigger");
22        wait_order(ev_A, ev_C, ev_B);
23        $display($time, "\tEvent's triggered Inorder");
24      end
25    join
26  end
27 endmodule

```

(b) What do you mean by overriding of a method while extending a sub-class from a base class? Discuss with a suitable example. Also, discuss about an abstract class and its importance.

2. For the following code given for semaphore, discuss about the displayed output. Briefly explain your result (point wise) by mentioning proper reason.

```

1 module tb_top;
2   semaphore room_key;
3   initial begin
4     room_key = new (2); // Two rooms available
5     fork
6       personX();
7       personY();
8       #2 personZ();
9     join_none
10  end
11  task getRoom(string name);
12    $display("[%0t] %s trying to get the room", $time, name);
13    room_key.get(1);
14    $display("[%0t] %s got the room", $time, name);
15  endtask
16  task putRoom(string name);
17    $display("[%0t] %s leaving the room", $time, name);
18    room_key.put(1);
19    $display("[%0t] %s released the room", $time, name);
20  endtask
21  task personX;
22    getRoom("PersonX");
23    #10 putRoom("PersonX");
24  endtask
25  task personY;
26    getRoom("PersonY");
27    #10 putRoom("PersonY");
28  endtask
29  task personZ;
30    getRoom("PersonZ");
31    #10 putRoom("PersonZ");
32  endtask
33 endmodule
34

```

3. Modify the given code to replace the `join_any` with `join` and observe the behaviour. Explain how `join` affects execution compared to `join_any`.

```

1 module tb;
2   initial begin
3     $display ("%0t] Main Thread: Fork join_any going to start", $time);
4     fork
5       fork
6         #50 $display ("%0t] Thread1_0 ...", $time);
7         #70 $display ("%0t] Thread1_1 ...", $time);
8         begin
9           #10 $display ("%0t] Thread1_2 ...", $time);
10          #100 $display ("%0t] Thread1_2 finished", $time);
11        end
12      join_any
13      begin
14        #5 $display ("%0t] Thread2 ...", $time);
15        #10 $display ("%0t] Thread2 finished", $time);
16      end
17      #20 $display ("%0t] Thread3 finished", $time);
18    join_any
19    $display ("%0t] Main Thread: Fork join_any has finished", $time);
20  end
21 endmodule
22

```

4. (a) For the following code, briefly discuss about each part of the code and predict the values of “a” by considering constraint. Mention the proper reason as well.

```

1 class packet;
2   rand bit [3:0] a;
3   string str;
4   constraint start_a {if(str = small) a > 5 else a < 5; }
5 endclass
6
7 module fstr;
8   initial begin
9     packet pkt;
10    pkt = new();
11    pkt.str= "small";
12    repeat (10) begin
13      pkt.randomize();
14      $display( "\t a =%d",pkt.a); end
15  end endmodule
16
17

```

- (b) With the help of suitable example, discuss about how one of the important features of OOPs i.e. data hiding is achieved in SystemVerilog. For clarity, take the help of local and protected access control concept.
5. (a) Modify the given code to use a priority if-else (instead of unique if) and analyse the behaviour when $x = 6$. Add an else clause for the second conditional block and observe the results. Discuss the difference between these two.

```

1 module tb;
2   int x = 6;
3
4   initial begin
5     if (x == 3)
6       $display("x is %0d", x);
7     else if (x == 5)
8       $display("x is %0d", x);
9     else
10      $display("x is neither 3 nor 5");|
11     if (x == 3)
12       $display("x is %0d", x);
13     else if (x == 5)
14       $display("x is %0d", x);
15     else
16       $display("This is the added else statement: x is %0d", x);
17   end
18 endmodule
19

```

- (b) How structure is different than array, discuss with the help of appropriate examples.
6. (a) What is the purpose of a monitor in a SystemVerilog UVM testbench? Show with a block diagram the interaction between monitor and other components in the verification environment.
- (b) Compare and contrast the roles of sequencer and driver in the UVM environment. How does randomization influence the effectiveness of the sequencer?
7. Write a SystemVerilog program to create a dynamic array containing the first 12 even numbers starting from 4. Using built-in methods and conditional logic, perform the following tasks: (a) Store and display all the elements that are multiples of 4 but not divisible by 8. (b) Count and display how many elements are greater than 10, and double those values in the original array. (c) Store the square of each element into a new dynamic array, and display both original and squared arrays.
8. There are two strings: str1 = "HDL is powerful" and str2 = "SystemVerilog enhances productivity". Write a SV program to do the following: (a) Reverse both strings. (b) Count and compare the number of consonants in each string (c) Display the difference in their consonant count.
9. Write a SV program using enum to define a type called State with values: IDLE = 0, START = 1, EXECUTE = 3, STOP = 5, ERROR = 7. (a) Use a foreach loop to print the name and value of each state. (b) Check whether the value "5" is defined in the enum. (c) Display a warning if ERROR state is active.
10. (a) Differentiate between static and dynamic arrays in SystemVerilog using a suitable example with proper code. (b) Given two queues q1 = {10, 15, 20, 25} and q2 = {5, 10, 15, 20}, write a SV program to perform the following tasks: (1) Create a new queue q3 by extracting the last two elements of q1 and the first two elements of q2. (2) Insert the number 100 at the second position of q3, and remove the last element. (3) Create another queue q4 by merging alternate elements from q1 and q2. (4) Display the contents of q1, q2, q3, and q4 after each operation.
11. Write a SV program to create a dynamic array containing the first 10 multiples of 3 starting from 6. Using built-in methods and conditional logic, perform the following tasks: a. Store and

display all the elements that are divisible by 6 but not by 9. Count and display how many elements are less than 25, and increment those values by 5 in the original array.

12. Write a SV program using “enum” to define a type called “State” with values: IDLE = 0, START = 1, EXECUTE = 3, STOP = 5, ERROR = 7. (a) Use a “foreach” loop to print the name and value of each state. (b) Check whether the value “5” is defined in the enum. (c) Display a warning if ERROR state is active.

13. Briefly discuss the role of a monitor while configuring a layered testbench to verify a DUT. Clearly explain the input and output flow for the “Driver” component. Also, describe the System Verilog features that make it preferable over Verilog HDL when the main objective is functional verification of a design.

14. Write a SV program using enum to define a type called Priority with values: LOW = 1, MEDIUM = 4, HIGH = 6, CRITICAL = 9, INVALID = 15. **(a)** Use a foreach loop to print the name and value of each priority level. **(b)** Check whether the value “6” is defined in the enum. **(c)** Display a warning if the current task priority is CRITICAL.

15. Design a SV program that keeps track of quiz scores for students. Use an associative array where each key is a student's name (string) and the value is their quiz score (int). Traverse the array using a loop to determine and display the names of the students with the highest and lowest scores.

16. Given two queues Q1 = {12, 18, 20, 28} and Q2 = {6, 12, 18, 24}, write a SV program to perform the following tasks: (1) Create a new queue Q3 by extracting the first two elements of Q1 and the last element of Q2. (2) Insert the number 60 at the third position of Q3, and remove the last element. (3) Create another queue Q4 by merging alternate elements from Q1 and Q2. (4) Display the contents of Q1, Q2, Q3, and Q4 after each operation.

17. Write a program to check whether the sum of two integers is a prime number or not by defining a function. Perform this functionality by calling the function and passing two values as arguments, then display the result after returning it from the function. Perform the same operation using a task in place of the function. Assume any two numbers as per your choice.

18. Define a class vehicle with three data members like model (string), color (string), and mileage (integer). Define a parameterized constructor to create two objects with data like {"Swift", "White", 22} and {"Creta", "Black", 18}. Define a method display inside the class to print all the data of these two objects. Perform the same task using a default constructor and another method assign data to assign values to the data members of these two classes.

19. (a) Differentiate between packed and unpacked arrays in System Verilog using a suitable example with proper code.

(b) Compare and contrast the roles of monitor and sequencer in the UVM environment. How does randomization influence the effectiveness of the sequencer?

20. Write a SV program using qa = {3, 6, 9, 12} and qb = {2, 4, 6, 8}. Create qc using the first 3 elements of qa and last of qb. Insert 50 at the beginning of qc and delete its 3rd element. Create qd by merging qa and qb in reverse, alternating elements. Display all queues after each step.

21. Write a SV program to reverse a string by defining a function. Call the function by passing a string as an argument and display the reversed string returned by it. Perform the same operation using a task instead of a function. Assume any string value of your choice.
22. In a restaurant, there are only three tables available for customers. Multiple customers arrive at different times and wait for a table to be free. Use semaphores to control the access to tables. Analyse the sequence of table allocations and releases.

```

1 module tb_restaurant;
2     semaphore tables;
3     initial begin
4         tables = new(3);
5         fork
6             customer("Customer1");
7             #3 customer("Customer2");
8             #5 customer("Customer3");
9             #8 customer("Customer4");
10        join_none
11    end
12    task customer(string name);
13    begin
14        $display("[%0t] %s is waiting for a table", $time, name);
15        tables.get(1); // Request 1 table
16        $display("[%0t] %s got a table", $time, name);
17        #10; // Dining time
18        $display("[%0t] %s is done dining", $time, name);
19        tables.put(1); // Release the table
20        $display("[%0t] %s left the table", $time, name);
21    end
22    endtask
23 endmodule

```

23. Briefly discuss about the role of driver while configuring a layered testbench to verify a DUT. Clearly discuss about input and output flow for “Driver” component. Which features of the System Verilog make it preferable over Verilog HDL when verification of a DUT is the main objective?
24. (a) With the help of suitable example, discuss about how one of the important features of OOPs i.e. data hiding is achieved in System Verilog. For clarity, take the help of local and protected access control concept.
(b) What do you mean by overriding of a method while extending a sub-class from a base class? Discuss with a suitable example. Also, discuss about an abstract class and its importance.
25. For the following code given for events, discuss about the displayed output. Briefly explain your result (point wise) by mentioning proper reason.

```

1 module events_example;
2   event ev_A, ev_B, ev_C;
3   initial begin
4     fork
5       begin
6         #12
7         $display($time, "\t triggering The Event ev_A");
8         ->ev_A;
9       end
10      begin
11        #5
12        $display($time, "\t triggering The Event ev_C");
13        ->ev_C;
14      end
15      begin
16        #8
17        $display($time, "\t triggering The Event ev_B");
18        ->ev_B;
19      end
20      begin
21        $display($time, "\t waiting for the Event to trigger");
22        wait_order(ev_A, ev_C, ev_B);
23        $display($time, "\tEvent's triggered Inorder");
24      end
25    join
26  end
27 endmodule

```

26. Evaluate the functional coverage for following code if randomly generated values of data are taken out of {1, 15, 20 and 22}. Briefly discuss about the process of evaluation.

```

1 module tb;
2   bit [4:0] mode;
3   covergroup cg;
4     coverpoint mode {
5       bins b1[] = {1, 7, 11, 26};
6       bins b2[] = {2:6};
7       bins b3[] = {[8:10], [15:19], 21};}
8   endgroup
9   cg cg_inst = new();
10  initial begin
11    for (int i = 0; i < 4; i++) begin
12      #10 mode = $random;
13      cg_inst.sample(); end
14    $display("coverage = %0.2f%%", cg_inst.get_inst_coverage());
15  end
16 endmodule

```

27. For the following code given for semaphore, discuss about the displayed output. Briefly explain your result (point wise) by mentioning proper reason.

```

1 module tb_top;
2   semaphore charger_key;
3   initial begin
4     charger_key = new(3);
5     fork
6       carA();
7       carB();
8       #2 carC();
9     join_none
10  end
11  task getCharger(string name);
12    $display("[%0t] %s is waiting for a charger...", $time, name);
13    charger_key.get(1);
14    $display("[%0t] %s acquired a charger", $time, name);
15  endtask
16  task putCharger(string name);
17    $display("[%0t] %s finished charging", $time, name);
18    charger_key.put(1);
19    $display("[%0t] %s released the charger", $time, name);
20  endtask
21  task carA;
22    getCharger("CarA");
23    #10 putCharger("CarA");
24  endtask
25  task carB;
26    getCharger("CarB");
27    #10 putCharger("CarB");
28  endtask
29  task carC;
30    getCharger("CarC");
31    #10 putCharger("CarC");
32  endtask
33 endmodule

```

28. For the following code given for events, discuss about the displayed output. Briefly explain your result (point wise) by mentioning proper reason.

```

1 module machine_startup;
2   event machine1_start, machine2_start, machine3_start;
3   initial begin
4     fork
5       begin
6         #3 $display("[%0t] Triggering Machine 2 Start", $time);
7         ->machine2_start;
8       end
9       begin
10        #1 $display("[%0t] Triggering Machine 3 Start", $time);
11        ->machine3_start;
12      end
13      begin
14        #5 $display("[%0t] Triggering Machine 1 Start", $time);
15        ->machine1_start;
16      end
17      begin
18        $display("[%0t] Monitoring startup sequence...", $time);
19        wait_order(machine3_start, machine2_start, machine1_start);
20        $display("[%0t] All machines started in correct order!", $time);
21      end
22    join
23  end
24 endmodule

```

29. With the help of waveform (clk showing pass and fail), write down details about where the given assertion is going to be pass and where it will be fail. Briefly discuss the reason.

```

1 module ass_tb3;
2     bit clk,a,b;
3     always #6| clk = clk;
4     initial begin
5         a=1; b=1;
6         #18 a=0; b=1;
7         #12 a=1; b=1;
8         #12 a=0; b=0;
9         #12 a=1; b=0;
10        #12 a=0; b=1;
11        #12;
12        $finish ; end
13
14    property p2;
15        @(posedge clk) b |-> ($past(a,2)==1);
16    endproperty
17
18    ast:assert property (p2);
19
20    end
21 endmodule

```

30. Evaluate the functional coverage for the following code if randomly generated values of opcode are taken out of {0, 2, 4, 6, 8}. Define a covergroup to check whether the values fall in the range [4:7]. Simulate for 5 random samples and display the final coverage

```

1 module tb;
2     bit [3:0] opcode;
3     covergroup cg_opcode(ref bit [3:0] op);
4         coverpoint op {
5             bins range4_7[] = {[4:7]};
6         }
7     endgroup
8     cg_opcode cg_inst;
9     initial begin
10        cg_inst = new(opcode);
11        for (int i = 0; i < 5; i++) begin
12            #10 opcode = $urandom_range(0, 8);
13            $display("[%0t] opcode = 0x%0h", $time, opcode);
14            cg_inst.sample();
15        end
16        $display("coverage = %0.2f%%", cg_inst.get_inst_coverage());
17    end
18 endmodule

```

31. Based on signal transitions and assertion semantics, identify the clock cycles where the assertion fails or passes, and explain the reason for each outcome.

```

1 module tb_handshake;
2     bit clk, req, ack;
3     always #5 clk = ~clk;
4     initial begin
5         clk = 0; req = 0; ack = 0;
6         #10 req = 1;
7         #10 ack = 1;
8         #10 req = 0; ack = 0;
9         #10 req = 1;
10        #30 ack = 1;
11        #10 $finish;
12    end
13    property p_handshake;
14        @(posedge clk) req |-> ##[1:2] ack;
15    endproperty
16    a_handshake: assert property(p_handshake);
17 endmodule

```

32. Modify the given code to implement a health condition checker using priority if. Assign a temperature value and check for “High Fever” (above 102), “Moderate Fever” (100–102),

“Normal” (98–99), else “Below Normal”. Test with temp = 98 and observe the output. Then change temp = 96 and retest to verify the else clause.

```
1 module tb;
2   int temp = 98;
3   initial begin
4     priority if (temp > 102)
5       $display("High Fever");
6     else if (temp >= 100)
7       $display("Moderate Fever");
8     else if (temp >= 98)
9       $display("Normal");
10    else
11      $display("Below Normal");
12  end
13 endmodule
```

33. Modify the given code to implement a grading system using priority if. Assign a score and check for Grade A (90–100), B (80–89), C (70–79), else Fail. Test with score = 75 and observe the output. Then add an else clause and retest.

```
1 module tb;
2   int score = 75;
3   initial begin
4     priority if (score >= 90)
5       $display("Grade A");
6     else if (score >= 80)
7       $display("Grade B");
8     else if (score >= 70)
9       $display("Grade C");
10    else
11      $display("Fail");
12  end
13 endmodule
```

34. (a) Differentiate between static and dynamic arrays in System Verilog using a suitable example with proper code.
(b) Compare and contrast the roles of sequencer and driver in the UVM environment. How does randomization influence the effectiveness of the sequencer?